Benchmarking Multi-View Tree-Level Oil Palm Bunch Counting Under a Fixed Detector

Anonymous Authors

Abstract—In oil palm plantations, each harvest cycle is planned from a per-tree count of fruit bunches grouped by maturity class, since each class ripens on a different schedule. Multi-view imaging recovers bunches hidden by occlusion, but the same bunch then reappears across several side views, so per-image detections must be aggregated into a per-tree count rather than summed. How well different counting methods perform this tree-level aggregation has not been benchmarked for oil palm. This paper evaluates five regression counters and eight feature configurations on 953 oil palm trees and 3,992 side-view images, under two detection conditions: ground-truth detections, which measure the counters in isolation, and a fixed YOLO26m detector, which reflects a deployed pipeline. With ground-truth detections, the best counter reaches 98.05% accuracy at the class level and 92.20% at the whole-tree level. Under the fixed detector, both fall sharply, to 77.48% and 32.62%. This gap of 20.57 and 59.58 points respectively persists across all five counters and eight feature configurations. Although the study sets out to compare counting methods, the dominant finding is that detector output quality, more than the choice of counter, limits deployed accuracy.

Keywords—Oil palm, fresh fruit bunch, black bunch census, multi-view counting, object detection, YOLO, tree-level regression, agricultural computer vision.

I. Introduction

Harvest planning in commercial oil palm plantations depends on knowing, for each surveyed tree, how many fruit bunches belong to each operational maturity class. The Black Bunch Census (BBC) formalises this requirement into a four-class count vector, B1 through B4, where each class carries distinct timing and logistics implications [1]. Automating BBC through image-based counting would reduce the cost and subjectivity of manual field surveys. An automated census is therefore useful only if it recovers the full per-class breakdown for each tree.

Tree-level counting is geometrically harder than image-level detection [2]. Bunches surround the full circumference of an oil palm, so a single image misses bunches due to occlusion, viewing angle, or overlapping fronds. Multi-view acquisition reduces missed bunches by photographing each tree from four to eight side views, but the same physical bunch then appears in several images. Fig. 1 illustrates the effect: a single B3 bunch is visible in three adjacent side views of the same tree. Naive summation of per-image detections counts appearances rather than bunch identities and is biased upward on this dataset.

Multi-view deduplication has been studied across several crops through geometric fusion, sequential scan, and neural reconstruction. Mango counting systems associated detections across sequential passes using navigation data and canopy geometry [3], and apple pipelines built row-level maps from overlapping two-side scans [4]. Comparative evaluation of multi-view apple pipelines shows that counting errors typically originate in detection quality rather than the aggregation step [5]. Sparse and calibrated viewpoints have been addressed through correspondence-free triangulation [6] and few-view 3D reconstruction [7], and recent neural-field methods count from unstructured photographs in a shared 3D representation [8]. Most of these systems depend on dense sequential overlap, a moving sensor platform, or calibrated geometry, and none isolates detector quality from counter quality as separate error sources.

Regression-based aggregation maps detection statistics to a tree-level count vector, an approach well established in agricultural vision [2], [4], [5]. YOLO-style single-stage detectors are now the standard front-end for such pipelines [9], [10]. A counter is typically evaluated only on the detector outputs available during the study, so its reported accuracy reflects that detector's limitations as much as the counter's own quality. Benchmarking counting methods on their own terms therefore calls for a detection-controlled condition alongside the realistic one, so that the counter's ceiling and its deployed performance can both be read.

Oil palm counting has received less systematic study than apple or mango. Detectors for fresh fruit bunches (FFB) and maturity classification have reached production-level accuracy on ripe bunches but show recurring weaknesses on partially occluded and maturity-ambiguous classes [15], with both two-stage and YOLO-style architectures explored [11]–[14]. The closest public oil palm resource is a video dataset annotated at the frame level for detection [16]. Tree-level counting labels are absent, and no counting evaluation has been reported on it. No existing benchmark evaluates tree-level counting strategies for oil palm under both controlled and realistic detection conditions.

This paper closes that gap by benchmarking tree-level multi-view BBC counting under two detection conditions on the SawitMVC dataset (953 trees, 3,992 images) [17]. The first condition uses ground-truth detections to isolate counter performance from detector error, and the second uses a fixed YOLO26m detector to reflect a realistic deployed pipeline. Five regression models and eight feature configurations are evaluated within each condition. The paper makes three contributions: (1) a systematic bench-

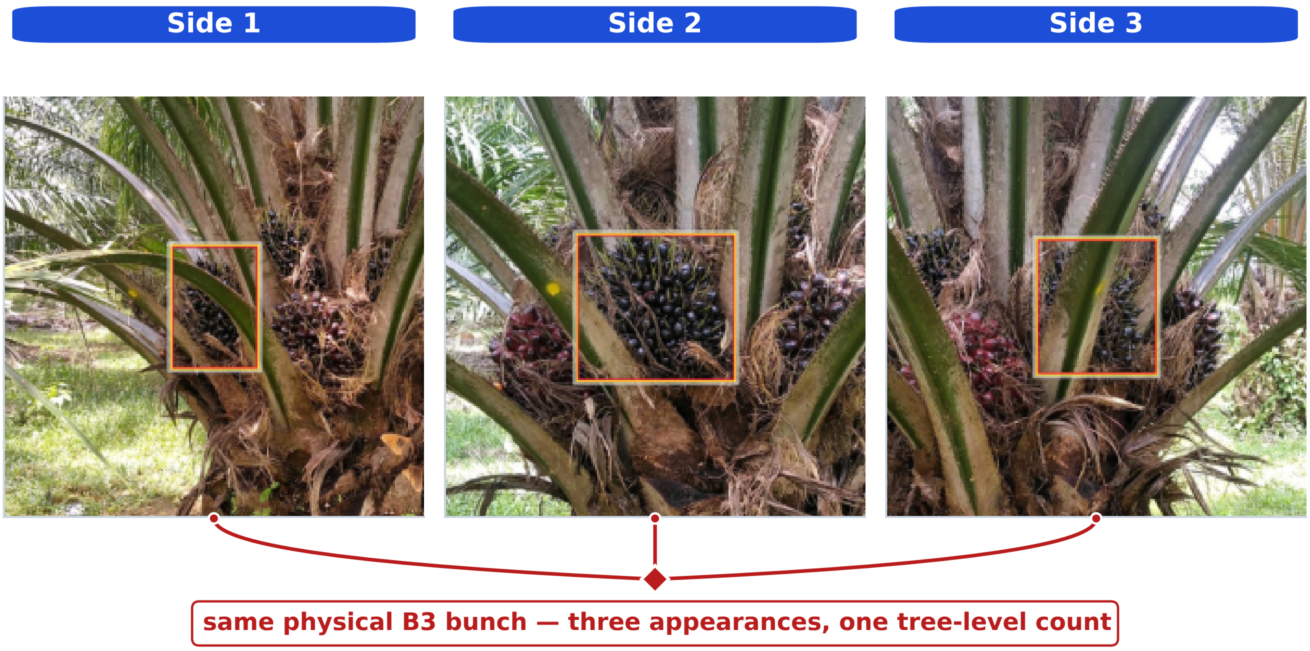

Fig. 1. A single physical B3 bunch is visible across three adjacent side views. If every per-image detection is counted, three appearances are added for one bunch. Tree-level counting must therefore aggregate evidence across views rather than sum it.

mark of tree-level bunch counting under two detection conditions, establishing an accuracy ceiling and a deployment baseline on the same dataset; (2) a feature ablation under Ridge showing that enriched representations recover only a marginal fraction of the detector-driven gap; and (3) evidence that the primary performance bottleneck is detector output quality rather than the choice of counting model.

II. Methodology

A. Dataset and Task Definition

The benchmark uses the SawitMVC multi-view dataset: 953 oil palm trees, 3,992 side-view images, four maturity classes (B1, B2, B3, and B4), and 4 to 8 side views per tree. The fixed split contains 716 training trees, 96 validation trees, and 141 test trees. Dataset construction, annotation protocol, and per-class composition are documented in [17].

For tree i, the ground-truth target is the count vector

$$\mathbf{y}_i = [\, y_{i,B1},\ y_{i,B2},\ y_{i,B3},\ y_{i,B4} \,], \tag{1}$$

and the prediction is $\hat{\mathbf{y}}_i = [\, \hat{y}_{i,B1}, \ldots, \hat{y}_{i,B4} \,]$. The evaluation unit is the tree, not the individual image: per-image appearance counts are not valid BBC outputs.

Let $a_{i,c}$ denote the number of detected appearances of class c across all views of tree i. The naive multi-view estimator

$$\hat{y}_{i,c}^{\text{naive}} = a_{i,c} \tag{2}$$

is biased upward whenever a bunch is visible from more than one side. Under GT annotations, this estimator reaches only 50.00% Class ±1 Acc and 6.38% Tree ±1 Acc on the test split, which motivates learned tree-level aggregation rather than direct summation.

B. Detection Conditions

The benchmark isolates detector error from counter error by evaluating the same counters under two detection conditions, illustrated in Fig. 2.

In the GT detection setting, the feature extractor reads ground-truth bounding boxes and classes directly from the annotation files. Counters trained on these features upper-bound how well a tree-level counter can perform when detections are accurate. In the fixed-detector setting, the feature extractor reads cached YOLO26m [10] predictions from the released y26mv2 checkpoint. The detector was trained on the official 716-tree training split for 60 epochs (batch size 32, image size 640, patience 60, seed 42). Inference uses the repository default confidence threshold of 0.25 and the standard Ultralytics post-processing settings. A single detector checkpoint is used for all counter and feature configurations, so any observed differences come from the counter or feature representation rather than detector retraining. Among the detector architectures and training configurations evaluated, validation performance was similar, so the y26mv2 checkpoint was selected as a representative fixed detector. A deeper detector study,

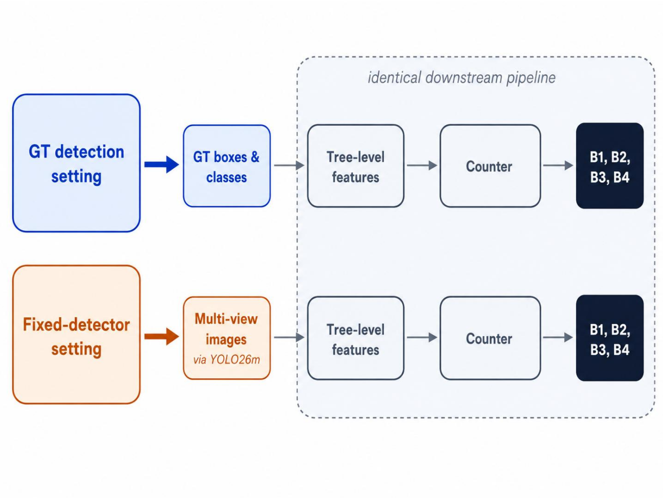

Fig. 2. Two detection conditions used in the benchmark. Top: the GT detection setting supplies ground-truth boxes and classes to the tree-level feature extractor, so detector output quality is removed as an error source. Bottom: the fixed-detector setting supplies cached YOLO26m outputs, so detector misses and class confusions enter the pipeline before counting.

including alternative architectures and depth-augmented inputs, is planned as future work.

TABLE I
YOLO26m validation detection performance, reported under the COCO mAP50 convention [18].

Class	mAP50	Recall
B1	0.746	0.801
B2	0.425	0.433
B3	0.550	0.656
B4	0.363	0.389
Overall	0.521	0.570

The per-class weakness on B4 and the moderate recall on B3 in Table I are revisited in Section III-D.

C. Feature Vectors

Given either set of detections, observations from all views of a tree are aggregated into a fixed-length feature vector. The 13-dimensional baseline F_0 contains, for each class c, the total count s_c, the per-side maximum m_c, and the per-side mean μ_c, plus the number of available views n_{views}:

$$F_0 = [\, s_{B1:B4},\ m_{B1:B4},\ \mu_{B1:B4},\ n_{\text{views}} \,]. \tag{3}$$

Three optional groups extend F_0. The confidence group (20-dim) contains five per-class statistics: confidence sum, mean, max, count above 0.5, and count above 0.6. Spatial position provides a separate signal: the mean normalized vertical centroid $\overline{cy}_c$ and mean bounding-box area $\overline{A}_c$ for each class form an 8-dimensional group that reflects the vertical separation of maturity classes on the tree. A 20-dimensional side-distribution group records per-side std, per-side min, coefficient of variation, number of sides with detections, and a consistency score for each class.

A cross-class composition group (6-dim), consisting of the total detection count, four class fractions, and a B3-vs-(B2+B3) mixture ratio, completes the 67-dimensional combined set F_{all}:

$$F_{\text{all}} = F_0 \cup \text{conf} \cup \text{spatial} \cup \text{distrib} \cup \text{composition}.$$

Eight feature sets are evaluated: F_0, F_0+conf, F_0+spatial, F_0+distrib, F_0+conf+spatial, F_0+conf+distrib, F_0+distrib+spatial, and F_{all}.

D. Counter Models and Evaluation Metrics

Three heuristic baselines are included as reference points. The naive sum sums all detected appearances per class (defined in Section II-A). The global divisor scales the naive sum by a fixed constant k fit to the training split, correcting for the average number of times a single bunch is observed across views. The visibility-adaptive divisor replaces the global constant with a per-tree factor that varies with the number of side views captured for that tree.

Five regression models are evaluated in both detection conditions: Linear Regression, Ridge [19], ElasticNet [20], SVM [21], and Random Forest [22]. Each counter maps a tree feature vector $\mathbf{x}_i$ to the count vector $\hat{\mathbf{y}}_i = f_\theta(\mathbf{x}_i)$.

All machine-learning counters are trained only on the 716-tree training split, with deterministic seed 42; continuous outputs are rounded to the nearest non-negative integer before metric computation. Linear Regression, Ridge, ElasticNet, and SVM use standardized features. Ridge uses RidgeCV over $\alpha \in \{0.01, 0.1, 1, 10, 100, 500\}$, ElasticNet uses MultiTaskElasticNetCV with 5-fold cross-validation and 3000 maximum iterations, SVM uses an RBF SVR in a multi-output wrapper with 3-fold GridSearchCV over $C \in \{0.1, 1, 10\}$ and $\gamma \in \{\text{scale}, 0.01, 0.1\}$, and Random Forest uses 200 trees, maximum depth 10, and random state 42.

All counter models are evaluated on the fixed 141-tree test split. Class-level ± 1 correctness for tree i and class c is

$$I^{\pm 1}_{i,c} = \begin{cases} 1, & |\hat{y}_{i,c} - y_{i,c}| \le 1, \\ 0, & \text{otherwise.} \end{cases} \tag{4}$$

Class ± 1 Acc averages this indicator over all test trees and classes:

$$\text{Class } \pm 1 \text{ Acc} = \frac{1}{4N} \sum_{i=1}^{N} \sum_{c=1}^{4} I^{\pm 1}_{i,c}. \tag{5}$$

Tree ± 1 Acc is stricter: a tree counts as correct only if all four class predictions are within ± 1 simultaneously,

$$\text{Tree } \pm 1 \text{ Acc} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}\left(\sum_{c=1}^{4} I^{\pm 1}_{i,c} = 4 \right). \tag{6}$$

Macro MAE is the mean absolute error averaged over trees and classes. Signed class bias is the per-class mean

$\hat{y}_{i,c} - y_{i,c}$; positive values indicate over-counting, negative values under-counting. Bias is reported alongside accuracy because operational aggregate yield estimates are sensitive to directional error rather than absolute error alone.[1]

III. Results and Discussion

The results are organised around the two detection conditions. Section III-A estimates the counter ceiling under GT detections, Section III-B measures the deployed fixed-detector setting, Section III-C tests whether richer features recover the loss, and Section III-D compares the two conditions per class.

A. GT Detection Setting

Table II reports counting results when the counter receives GT detections. Naive appearance summation yields only 50.00% Class ±1 Acc and 6.38% Tree ±1 Acc. This confirms that duplicate visibility cannot be ignored and that direct summation is not a viable estimator. The global divisor and visibility-adaptive divisor recover most of the loss, and ElasticNet improves further to 98.05% Class ±1 Acc. Under accurate detections, the remaining counting problem is therefore simple enough for compact regression models.

TABLE II
Counting under the GT detection setting on the 141-tree test split.

Method	Type	Class ±1	Tree ±1	MAE
Naive sum	Heur.	50.00%	6.38%	2.142
Global divisor	Heur.	95.39%	85.11%	0.376
Visibility-adapt. divisor	Heur.	95.92%	87.23%	0.340
ElasticNet	ML	98.05%	92.20%	0.277
SVM	ML	97.87%	91.49%	0.266
Ridge	ML	97.70%	90.78%	0.275
Linear reg.	ML	97.52%	90.07%	0.277
Random forest	ML	95.92%	84.40%	0.365

The four best machine-learning counters (ElasticNet, SVM, Ridge, LR) sit within 0.6 percentage points of each other on Class ±1 Acc and within 2.2 points on Tree ±1 Acc. Random Forest is the only counter that underperforms in this setting. This narrow spread indicates that, when detector evidence is accurate, the choice among standard counter families is not the limiting factor.

B. Fixed-Detector Setting

Moving from GT to cached YOLO26m detections, the best F_0 result (ElasticNet, 76.42% Class ±1 Acc) trails the same counter under GT by more than 20 percentage points (Table III). All five counters remain within a 3.2 percentage-point window on Class ±1 Acc. The large drop across the two detection conditions, coupled with the small spread among counters, points to the detector output rather than the counter family as the main source of loss.

The per-class breakdown in Table IV shows that B1 stays near its GT ceiling while B3 falls to 55–56% Class ±1 Acc across all five counters. B2 and B4 sit at intermediate losses. This profile matches the detection recall in Table I: the weakest detector classes lose the most counting accuracy.

[1]Source code: https://anonymous.4open.science/r/Baseline-SawitMVC.

TABLE III
Fixed-detector counting with F0 features on the 141-tree test split.

Counter	Class ±1 Acc	Tree ±1 Acc	Macro MAE
ElasticNet	76.42%	29.79%	1.043
Ridge	76.06%	28.37%	1.053
Linear Regression	75.71%	30.50%	1.048
SVM	74.82%	29.08%	1.043
Random Forest	73.23%	26.95%	1.110

TABLE IV
Per-class Class ±1 Acc of fixed-detector counters with F0 features on the 141-tree test split.

Counter	B1	B2	B3	B4
ElasticNet	96.45%	80.14%	56.03%	73.05%
Ridge	95.74%	80.14%	56.03%	72.34%
Linear Regression	96.45%	79.43%	55.32%	71.63%
SVM	95.74%	76.60%	55.32%	71.63%
Random Forest	95.74%	73.76%	56.03%	67.38%

C. Feature Ablation

TABLE V
Ridge feature ablation in the fixed-detector setting.

Features	Dim	Class ±1	Tree ±1	MAE
F_0	13	76.06%	28.37%	1.053
F_0 + confidence	33	75.89%	29.79%	1.059
F_0 + spatial	21	76.60%	30.50%	1.046
F_0 + side-distribution	33	74.82%	28.37%	1.060
F_0 + confidence + spatial	41	76.24%	29.08%	1.059
F_0 + confidence + side-distribution	53	76.42%	29.79%	1.060
F_0 + side-distribution + spatial	41	75.71%	30.50%	1.048
F_{all}	67	77.48%	32.62%	1.035

Whether enriched features can recover the detector-driven gap is tested by fixing Ridge and varying the feature configuration (Table V). F_0 starts at 76.06% Class ±1 Acc. Spatial features alone raise this to 76.60%; the vertical centroid and bounding-box area encode residual maturity position that the raw count statistics in F_0 do not capture directly. Confidence features slightly lower Class ±1 Acc (75.89% vs. 76.06%) but raise Tree ±1 Acc by 1.42 points. Adding side-distribution alone reduces Class ±1 Acc to 74.82%, below the F_0 baseline, so this group adds little under the current detector.

The full F_{all} set gives the best ablation result at 77.48% Class ±1 Acc and 32.62% Tree ±1 Acc. The gain over F_0 is 1.42 percentage points at class level and 4.25 points at tree level. This 1.42-point gain marks the ceiling of feature-side improvement under Ridge. The spread across the five counter families is separately small, within 3.2 points (Table III). The best feature set narrows only a small fraction of the gap between GT and fixed-detector conditions.

D. GT vs. Fixed-Detector Gap

The central comparison is the per-class gap between ideal detections and the fixed detector, shown with the signed bias of the best configuration in Fig. 3.

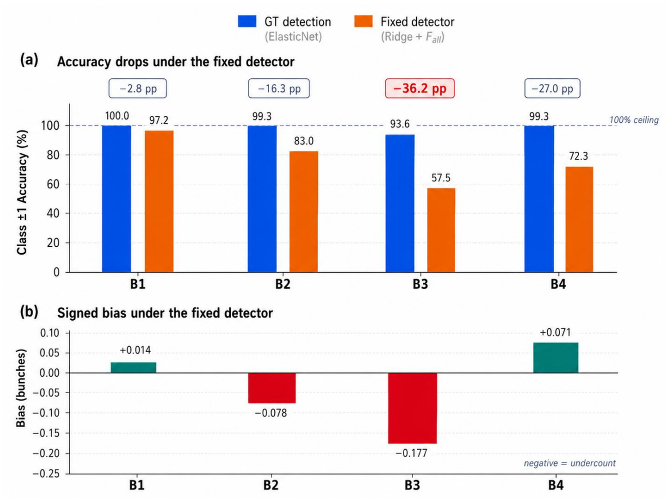

Fig. 3. Top: Per-class Class ±1 Acc for the best counter in each detection condition (ElasticNet under GT; Ridge + F_{all} under the fixed detector). Counting is near ceiling under GT detection but drops sharply under the fixed detector. Bottom: Per-class signed bias of Ridge + F_{all} in the fixed-detector setting. Negative values indicate systematic under-counting; the under-count is concentrated on B2 and especially B3.

The absolute gap is large, at 20.57 percentage points in Class ±1 Acc and 59.58 percentage points in Tree ±1 Acc (Table VI). The loss is not uniform across maturity classes (Fig. 3, top). B1 loses only ≈2.8 pp between the two conditions, while B2 loses ≈16.3 pp, B3 loses 36.2 pp, and B4 loses ≈27.0 pp. The classes that fall furthest, B3 and B4, are also the weakest in detector recall (Table I).

The bias plot adds direction to the accuracy gap. The fixed-detector Ridge + F_{all} pipeline under-counts B2 by 0.078 and B3 by 0.177 bunches per tree on average. B4 moves the other way, over-counted by 0.071 despite its low recall, a pattern consistent with B3→B4 class confusion rather than missed detections. Because plantation-level forecasts aggregate per-tree predictions across blocks, these directional errors carry into the next-cycle harvest estimate rather than cancel out as random noise.

TABLE VI
Consolidated test-set summary. Biases are ordered by class.

Config	Class ±1	Tree ±1	B1	B2	B3	B4
GT, ElasticNet	98.05%	92.20%	−.050	+.043	−.064	−.028
Fixed, Ridge + F_{all}	77.48%	32.62%	+.014	−.078	−.177	+.071

Two mechanisms explain why the gap resists counter-side fixes. A bunch the detector misses in every view leaves no evidence for any downstream counter to recover, so the recall-aligned part of the gap is intrinsic to the detector. The remaining part reflects class confusion at the B2/B3 and B3/B4 boundaries, which the cross-class composition features in F_{all} target directly yet narrow only marginally. The per-class gap therefore traces the detector recall profile in Table I more than the spread among counter families. This matches the apple-counting finding that errors trace mainly to detection quality [5].

These findings have a direct consequence for system design. Because the missing accuracy is information the detector never produced, no counter or feature combination tested here recovers more than 1.42 points of it at the class level. The highest-leverage intervention is therefore detector-side and class-specific, raising recall for B3 and improving class precision for B4 to remove the confusion that inflates its count.

Several limitations bound the scope of these findings. The benchmark uses a single dataset of 953 trees from two plantations in South Kalimantan, so absolute numbers may shift under other regions, varieties, or acquisition protocols. Only one detector checkpoint (YOLO26m) is tested, and a different detector family could change the per-class error profile. The tree-level counters treat each tree independently and ignore plantation-scale spatial or temporal regularities that operational forecasts sometimes exploit. The four-class B1 through B4 taxonomy also admits visual ambiguity at the B2/B3 and B3/B4 boundaries. Future work should prioritize B3/B4 recall, confidence calibration for counting, and explicit cross-view association or geometry-aware aggregation [6], [7], [23] rather than expanding the tree-level feature set further.

IV. Conclusion

This paper benchmarks multi-view tree-level oil palm FFB counting by evaluating the same counters under GT detections and cached YOLO26m outputs. With GT detections, standard regression counters achieve near-ceiling accuracy on the B1 through B4 count. With the fixed detector, neither additional feature groups nor alternative counter families narrow the gap appreciably. On SawitMVC under this detector checkpoint, the limiting factor is detector output quality more than the choice among the tested tree-level counters. Closing the gap will require a stronger detector, particularly higher recall for B3 and B4.

References

[1] M. Kerstan and T. Fairhurst, “Use of OMP for accurate crop forecasts,” Agrisoft Systems and Tropical Crop Consultants Limited. [Online]. Available: https://www.agrisoft-systems.com/wp-content/uploads/Papers/CropForecastsWithOMPBBC.pdf. [Accessed: May 2026].

[2] A. Koirala, K. B. Walsh, Z. Wang, and C. McCarthy, “Deep learning: method overview and review of use for fruit detection and yield estimation,” Computers and Electronics in Agriculture, vol. 162, pp. 219–234, Jul. 2019, doi: https://doi.org/10.1016/j.compag.2019.04.017.

[3] M. Stein, S. Bargoti, and J. Underwood, “Image based mango fruit detection, localisation and yield estimation using multiple view geometry,” Sensors, vol. 16, no. 11, Art. no. 1915, Nov. 2016, doi: https://doi.org/10.3390/s16111915.

[4] P. Roy, A. Kislay, P. A. Plonski, J. Luby, and V. Isler, “Vision-based preharvest yield mapping for apple orchards,” Computers and Electronics in Agriculture, vol. 164, Art. no. 104897, Sep. 2019, doi: https://doi.org/10.1016/j.compag.2019.104897.

[5] N. Häni, P. Roy, and V. Isler, “A comparative study of fruit detection and counting methods for yield mapping in apple orchards,” Journal of Field Robotics, vol. 37, no. 2, pp. 263–282, Mar. 2020, doi: https://doi.org/10.1002/rob.21902.

[6] M. Gaillard, B. Benes, M. C. Tross, and J. C. Schnable, “Multi-view triangulation without correspondences,” Computers and Electronics in Agriculture, vol. 206, Art. no. 107688, Mar. 2023, doi: https://doi.org/10.1016/j.compag.2023.107688.

[7] H. Freeman, E. Schneider, C. H. Kim, M. Lee, and G. Kantor, “3D reconstruction-based seed counting of sorghum panicles for agricultural inspection,” in Proc. IEEE International Conference on Robotics and Automation (ICRA), 2023, pp. 9594–9600, doi: https://doi.org/10.1109/ICRA48891.2023.10161400.

[8] L. Meyer, A.-T. Ardelean, T. Weyrich, and M. Stamminger, “FruitNeRF++: A generalized multi-fruit counting method utilizing contrastive learning and neural radiance fields,” in Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2025, pp. 1717–1724, doi: https://doi.org/10.1109/IROS60139.2025.11247341.

[9] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 7464–7475, doi: https://doi.org/10.1109/CVPR52729.2023.00721.

[10] G. Jocher and J. Qiu, “Ultralytics YOLO26,” Ultralytics, version 26.0.0, 2026. [Software]. Available: https://github.com/ultralytics/ultralytics.

[11] N. A. Prasetyo, Pranowo, and A. J. Santoso, “Automatic detection and calculation of palm oil fresh fruit bunches using Faster R-CNN,” International Journal of Applied Science and Engineering, vol. 17, no. 2, pp. 121–134, 2020, doi: https://doi.org/10.6703/IJASE.202005_17(2).121. [Online]. Available: https://gigvvy.com/journals/ijase/articles/ijase-202005-17-2-121

[12] J. W. Lai, H. R. Ramli, L. I. Ismail, and W. Z. W. Hasan, “Real-time detection of ripe oil palm fresh fruit bunch based on YOLOv4,” IEEE Access, vol. 10, pp. 95763–95770, 2022, doi: https://doi.org/10.1109/ACCESS.2022.3204762.

[13] M. Y. M. A. Mansour, K. D. Dambul, and K. Y. Choo, “Object detection algorithms for ripeness classification of oil palm fresh fruit bunch,” International Journal of Technology, vol. 13, no. 6, pp. 1326–1335, 2022, doi: https://doi.org/10.14716/ijtech.v13i6.5932.

[14] M. H. Junos, A. S. M. Khairuddin, S. Thannirmalai, and M. Dahari, “Automatic detection of oil palm fruits from UAV images using an improved YOLO model,” The Visual Computer, vol. 38, no. 7, pp. 2341–2355, 2022, doi: https://doi.org/10.1007/s00371-021-02116-3.

[15] M. G. Naftali, G. Hugo, and Suharjito, “Palm oil counter: State-of-the-art deep learning models for detection and counting in plantations,” IEEE Access, vol. 12, pp. 90395–90417, 2024, doi: https://doi.org/10.1109/ACCESS.2024.3419835.

[16] Suharjito, M. G. Naftali, G. Hugo, M. R. A. Priyadi, M. Asrol, and D. N. Utama, “Oil palm fruits dataset in plantations for harvest estimation using digital census and smartphone,” Scientific Data, vol. 12, p. 972, Jun. 2025, doi: https://doi.org/10.1038/s41597-025-05227-x.

[17] F. Indriani et al., “SawitMVC: A Multi-View Oil Palm Fruit Bunch Dataset for Detection and Counting,” Zenodo, May 22, 2026, doi: https://doi.org/10.5281/zenodo.20336323.

[18] T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in Proc. European Conference on Computer Vision (ECCV), 2014, pp. 740–755, doi: https://doi.org/10.1007/978-3-319-10602-1_48.

[19] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” Technometrics, vol. 12, no. 1, pp. 55–67, 1970, doi: https://doi.org/10.1080/00401706.1970.10488634.

[20] H. Zou and T. Hastie, “Regularization and variable selection via the elastic net,” Journal of the Royal Statistical Society B, vol. 67, no. 2, pp. 301–320, 2005, doi: https://doi.org/10.1111/j.1467-9868.2005.00503.x.

[21] C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995, doi: https://doi.org/10.1007/BF00994018.

[22] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: https://doi.org/10.1023/A:1010933404324.

[23] X. Liu, S. W. Chen, C. Liu, S. S. Shivakumar, J. Das, C. J. Taylor, J. Underwood, and V. Kumar, “Monocular camera based fruit counting and mapping with semantic data association,” IEEE Robotics and Automation Letters, vol. 4, no. 3, pp. 2296–2303, Jul. 2019, doi: https://doi.org/10.1109/LRA.2019.2901987.